Title of the project :- Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Overview:-

Enforcing least-privilege access means giving users only the minimum level of access required to perform their tasks. This security principle reduces the risk of unauthorized access, data breaches, and internal threats by ensuring that users cannot access systems, applications, or data that are not necessary for their role. In modern organizations where employees work remotely and access applications from different locations and devices, implementing least-privilege access has become an essential part of cybersecurity strategies.

Zscaler Private Access (ZPA) is a cloud-based Zero Trust Network Access solution developed by Zscaler that helps organizations securely connect users to private applications without exposing the internal network. Unlike traditional VPN solutions that allow users to connect to the entire corporate network, ZPA provides secure, direct connections only to specific applications. This approach follows the Zero Trust model, where no user or device is trusted by default and every access request must be verified before permission is granted.

ZPA enforces least-privilege access by using identity-aware policies. These policies evaluate several factors such as the user's identity, role, device status, location, and the application being requested. Based on these conditions, ZPA determines whether the user should be granted access. For example, a finance employee may be allowed to access financial applications but restricted from development or HR systems. This ensures that users only interact with resources relevant to their responsibilities.

Another key feature of ZPA is application-specific access control. Instead of giving users access to the entire network, administrators create policies that allow access only to particular applications. This limits the potential damage if an attacker gains access to a user account because the attacker cannot move freely across the network to reach other systems. By segmenting applications and controlling access through policies, organizations can significantly reduce the risk of lateral movement within the network.

Overall, ZPA helps organizations implement a strong Zero Trust security model by continuously verifying users and enforcing strict access policies. By combining identity verification, device posture checks, and application-level segmentation, ZPA ensures that users receive only the access they truly need. This approach improves security, protects sensitive resources, and provides a safer way for employees to access private applications from anywhere.

List of Teammates

S.no	name	collage	contact

01	Susith R	Nandha arts and science college	Susithsusith2005@gmail.com
02	Thiruviga M	Nandha arts and science college	ktamilarasan116@gmail.com
03	Vaishavi B	Nandha arts and science college	vaishavib.23@gmail.com
04	Varshini G	Nandha arts and science college	varshinisudha2005@gmail.com
05	Tamilarasan K	Nandha arts and science college	ktamilarasan116@gmail.com

List of Vulnerability Table

S.no	Vulnerability Name	CWE/CVE - No
01	Broken Access Control	CWE-23
02	Security Misconfiguration	CWE-16
03	Injection	CWE-20
04	Cryptographic Failure	CWE-261
05	Software Supply Chain Failures	CWE-447

06	Insecure Design	CWE-73
07	Authentication Failures	CWE-287
08	Software or Data Integrity Failures	CWE-426
09	Security Logging & Alerting Failures	CWE-221
10	Mishandling of Exceptional Conditions	CWE-234

Report:

1. Vulnerability name: Relative Path Traversal

CWE: 23

OWASP category: A01:2025 Broken Access Control

Description

Relative Path Traversal (also called Directory Traversal) is a web security vulnerability that allows an attacker to access files and directories stored outside the intended folder of a web application.
Normally, applications restrict users to access only specific files (for example: images, documents, or configuration files). However, if the application does not properly validate user input, an attacker can manipulate the file path using special characters such as:
../
This sequence means “go up one directory level” in the file system.
For example:
https://example.com/download?file=report.pdf
If the application is vulnerable, an attacker might modify the request like this:
https://example.com/download?file=../../../../etc/passwd
Here, the ../ sequences move up multiple directory levels, potentially allowing the attacker to access sensitive system files, configuration files, or application source code.
Common targets of path traversal attacks include:

- System configuration files
- Application source code

- Password files
- Database credentials
- Private user data

This vulnerability usually occurs when applications directly use user input in file path operations without proper validation or sanitization.

Business Impact

Relative Path Traversal vulnerabilities can have serious consequences for organizations, including:

1. Data Breach
   Attackers may gain access to sensitive files such as user data, passwords, API keys, and database credentials, leading to confidential information leaks.

2. System Compromise
   If attackers access critical configuration files, they may gain deeper control over the server or application, potentially leading to further exploitation.

3. Loss of Customer Trust
   Exposure of sensitive data can damage a company's reputation and reduce customer confidence in the organization's security practices.

4. Financial Loss
   Data breaches and system compromise may lead to financial penalties, legal costs, and recovery expenses.

5. Compliance Violations
   Organizations handling personal or financial data may violate regulations such as GDPR, PCI DSS, or HIPAA, resulting in fines and regulatory action.

6. Service Disruption
   Attackers may access or modify important application files, causing application failure or downtime.

2. Vulnerability name: Configuration

CWE: 16

OWASP category: A02:2025 Security Misconfiguration

Description

Configuration refers to the process of setting up system parameters, application settings, server options, and security controls that determine how software, hardware, or networks operate. In information systems, configuration includes settings for servers, databases, operating systems, applications, APIs, and cloud services.
A security misconfiguration occurs when these settings are incorrectly configured, left as default, or not properly secured, which can create vulnerabilities that attackers can exploit. Configuration issues are one of the most common causes of security breaches in modern systems.
Examples of configuration elements include:

- Server configuration files
- Database settings
- Network security rules
- Application permissions
- Cloud storage access controls
- API configurations
- Authentication and authorization settings

When systems are deployed without secure configuration, they may expose sensitive information, unnecessary services, debug interfaces, or administrative access.
Common examples of configuration vulnerabilities include:
1. Default Credentials
Systems may be installed with default usernames and passwords such as:
admin / admin root
/ password
If these are not changed, attackers can easily gain unauthorized access.
2. Unnecessary Services Enabled
Servers may run services that are not required for the application, increasing the attack surface.
3. Publicly Accessible Sensitive Data
Improper configuration may expose sensitive files or databases to the public internet.
Example:
Public cloud storage bucket with customer data
4. Debug or Development Settings Enabled
Applications sometimes run in debug mode, revealing internal information such as:

- File paths
- Server configuration
- Error messages
- Database queries

5. Incorrect Access Permissions
Files or directories may have permissions that allow unauthorized users to read or modify them.
6. Missing Security Headers or Encryption Settings
Improper configuration may disable important security protections like:

- HTTPS encryption
- Secure cookies
- Content security policies

Because configuration settings control how systems behave, even a small mistake can create major security vulnerabilities.

Business Impact

Configuration vulnerabilities can have serious consequences for organizations.

1. Unauthorized Access to Systems
   Improper configuration may allow attackers to gain access to servers, databases, or administrative interfaces.

2. Data Breaches
   Sensitive information such as customer data, financial records, and internal documents may become publicly accessible.

3. Financial Loss
   Organizations may face costs related to:

- Incident response
- Legal penalties
- Compensation for affected customers
- Security remediation

4. Regulatory and Compliance Violations
   Companies handling sensitive data may violate regulatory standards such as:

- GDPR
- PCI DSS
- HIPAA

This can lead to fines, audits, and legal consequences.

5. Service Disruption
   Misconfigured systems may cause application crashes, downtime, or operational failures, affecting business services.

6. Reputation Damage
   Security incidents caused by configuration errors can harm an organization's reputation and reduce customer trust.

7. Intellectual Property Exposure
   Improper configuration may expose source code, internal documents, or proprietary algorithms, leading to loss of competitive advantage.

3. Vulnerability name: Improper Input Validation

CWE: 20

OWASP category: A05:2025 Injection

Description

Improper Input Validation is a security vulnerability that occurs when a system or application fails to properly check, filter, or sanitize user input before processing it. Input validation ensures that the data entered by users meets the expected format, type, length, and range.
Most applications accept input from users through:

- Login forms
- Search fields
- File uploads
- Web forms
- API requests
- URL parameters

If the application does not validate these inputs correctly, attackers can provide malicious or unexpected data to manipulate the system.
For example, a login form expects a username and password. If the application does not validate the input, an attacker could insert malicious code or commands that the system processes unintentionally. Example of normal input:
Username: user123
Password: mypassword
Example of malicious input:
Username: admin' OR '1'='1
This type of input may allow attackers to bypass authentication in poorly designed systems.
Improper input validation can lead to many other security vulnerabilities, including:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- Path Traversal
- Buffer Overflow

Because user input enters the system from untrusted sources, validating it is essential for maintaining application security.

Business Impact

Improper Input Validation can have major consequences for organizations.
1. Unauthorized Access
Attackers may bypass login systems and gain access to restricted accounts or administrative privileges.
2. Data Breach
Sensitive information such as customer data, financial records, and internal documents may be exposed.
3. System Compromise
Attackers may execute malicious commands on servers, potentially taking full control of the system.

4. Financial Loss
Security incidents can lead to financial damage through fraud, system repair costs, and legal penalties.
5. Regulatory Compliance Violations
Organizations may violate security regulations if sensitive data is exposed due to poor input validation.
6. Service Disruption
Attackers may manipulate input to crash systems or disrupt services.
7. Reputation Damage
Customers may lose trust in an organization if security vulnerabilities lead to data leaks or system compromise.

4. Vulnerability name: Weak Encoding for Password

CWE: 261

OWASP category: A04:2025 Cryptographic Failures

Description

Weak Encoding for Password refers to a security vulnerability that occurs when passwords are stored or transmitted using weak encoding or reversible encoding techniques instead of strong cryptographic hashing methods. Encoding is not designed for security; it only converts data into another format for easy storage or transmission.
Many systems incorrectly store passwords using simple encoding methods such as:

- Base64 encoding
- Hex encoding
- Simple encryption without proper key protection

These encoding methods are easily reversible, meaning attackers can quickly decode the stored data and retrieve the original password.
For example, consider a password:
Password: admin123
If it is encoded using Base64, it becomes:
YWRtaW4xMjM=
However, this encoding can easily be reversed back to the original password using common decoding tools. Because of this, encoding does not provide real security protection for passwords.
Secure systems should use strong hashing algorithms such as:

- bcrypt
- Argon2
- PBKDF2
- SHA-256 with salting

Hashing converts the password into a fixed-length string that cannot be easily reversed. Additionally, a salt value should be added before hashing to protect against attacks such as rainbow table attacks.
Example of secure hashing process:
Password + Salt → Hash Function → Secure Hash

If a system uses weak encoding instead of proper hashing, attackers who gain access to the database can easily recover user passwords.

Business Impact

Weak password encoding can cause serious damage to organizations.
1. Account Compromise
Attackers may gain unauthorized access to user or administrator accounts.
2. Data Breaches
Compromised accounts may expose sensitive information such as personal data, financial records, or confidential documents.
3. Financial Loss
Companies may face costs related to fraud, security recovery, and customer compensation.
4. Regulatory Violations
Organizations handling personal or financial data may violate security regulations if passwords are not properly protected.
5. Reputation Damage
Customers may lose trust in the organization if their passwords are exposed due to weak security practices.
6. Increased Cyber Attacks
Attackers may use stolen credentials to perform further attacks within the organization's network.

5. Vulnerability name: Unimplemented or Unsupported Feature in UI

CWE: 447

OWASP category: A03:2025 Software Supply Chain Failures

Description

Unimplemented or Unsupported Features in the User Interface (UI) refer to situations where a feature appears to be available in the application's interface but is not fully implemented, properly supported, or securely handled in the backend system. This can create confusion for users and may expose security or functional weaknesses in the application.
In many software systems, developers design UI elements such as buttons, forms, menus, or links that suggest certain functionality. However, if these features are not fully implemented in the backend or are left incomplete during development, users may interact with them even though the system does not correctly process the request.
Examples of such features include:

- A button that performs no action when clicked
- A form that collects data but does not process or store it
- A menu option that leads to an error page

- A feature that is visible but not supported by the server

For example, a website might show a “Delete Account” button in the interface, but if the backend logic is not implemented, clicking it may produce an error or behave unexpectedly.
Example UI element:
[Delete Account]
If the backend does not support the action, the system may return an error such as:
Feature not implemented
In some cases, these unfinished UI elements are left in production due to development oversight, incomplete testing, or poor version control.
This issue is often related to poor software design, incomplete development cycles, or lack of proper quality assurance testing.

Business Impact

Unimplemented or unsupported UI features can negatively affect an organization in several ways.

Poor User Experience
Users may become frustrated when features shown in the interface do not work properly.

Loss of Customer Trust
Customers may lose confidence in the application if they encounter broken or incomplete functionality.

Security Risks
Incomplete features may expose vulnerabilities that attackers can exploit.

Operational Inefficiency
Support teams may receive more complaints and service requests from users.

Reputation Damage
Applications with visible unfinished features may appear unprofessional and unreliable.

Financial Loss
Poor user experience or security issues may lead to customer loss or additional development costs.

6. Vulnerability name: External Control of File Name or Path

CWE: 73

OWASP category: A06:2025 Insecure Design

Description

External Control of File Name or Path is a security vulnerability that occurs when an application allows user input or external data to determine the name or path of a file without proper validation or restrictions. This means that the system relies on untrusted input to locate, read, write, or modify files on the server.
In many applications, users can upload, download, or open files. The application may use the filename provided by the user to perform file operations. If the system does not properly validate this input, attackers can manipulate the filename or file path to access or modify unauthorized files.

For example, an application may allow users to download documents using a URL parameter: https://example.com/download?file=report.pdf
The application may internally process this request like this:
/var/www/files/report.pdf
If the application does not validate the file name, an attacker could manipulate the request:
https://example.com/download?file=../../etc/passwd
This may allow the attacker to access sensitive system files that are outside the intended directory.
Similarly, in file upload systems, attackers might control the file name or path and overwrite important files on the server.
Example:
Filename: ../../config/database.conf
If the application stores files using this name, it may overwrite important configuration files.
Because file systems are critical components of operating systems, allowing external control over file paths can lead to serious security vulnerabilities.

Business Impact

External Control of File Name or Path can have significant consequences for organizations.
Data Breaches
Attackers may gain access to confidential data such as user information, financial records, and internal documents.
System Compromise
Access to configuration files may allow attackers to control servers or databases.
Service Disruption
Critical system files may be deleted or modified, causing application failures.
Financial Loss
Organizations may incur costs for incident response, legal penalties, and system recovery.
Compliance Violations
Exposure of sensitive data may violate regulatory standards such as data protection laws.
Reputation Damage
Security breaches may harm the organization's credibility and customer trust.

7. Vulnerability name: Improper Authentication

CWE: 287

OWASP category: A07:2025 Authentication Failures

Description

Improper Authentication is a security vulnerability that occurs when a system fails to correctly verify the identity of users before granting access to protected resources.
Authentication is the process of confirming that a user is who they claim to be, usually through usernames, passwords, tokens, or biometric methods.
When authentication mechanisms are poorly implemented, attackers may be able to bypass login controls, impersonate legitimate users, or gain unauthorized access to sensitive systems and data.

Many web applications rely on authentication systems to protect features such as:

- User accounts
- Administrative dashboards
- Financial transactions
- Personal data access
- Internal management systems

If authentication controls are weak or incorrectly implemented, attackers may exploit these weaknesses to access restricted areas of the system.
Example of a normal login process:
User → Enter username and password → System verifies credentials → Access granted
If the authentication mechanism is flawed, the system may grant access without proper verification.

Business Impact

Improper authentication can cause severe damage to organizations.
Unauthorized Access
Attackers may gain access to user accounts, administrative systems, or internal resources.
Data Breaches
Sensitive information such as customer data, financial records, and confidential documents may be exposed.
Financial Fraud
Attackers may perform unauthorized transactions or manipulate financial systems.
Account Takeover
User accounts may be hijacked and used for malicious activities.
Regulatory Violations
Organizations may violate security regulations if authentication systems fail to protect sensitive information.
Reputation Damage
Security incidents related to authentication failures can reduce customer trust and harm the organization's reputation.

8. Vulnerability name: Untrusted Search Path

CWE: 426

OWASP category: A08:2025 Software or Data Integrity Failures

Description

Untrusted Search Path is a security vulnerability that occurs when an application loads or executes a program, library, or file from a search path that includes directories controlled by untrusted users. If the system searches for required files in unsafe locations, an attacker may place a malicious file with the same name in that location. The application may then unknowingly execute the malicious file instead of the legitimate one.
Operating systems often use a search path to locate executable files, libraries, or scripts. When a program tries to load a file without specifying its full path, the system searches through a list of directories until it finds a file with the requested name.

For example, if a program executes a command like:
run_program("backup.exe")
The operating system may search for backup.exe in several directories such as:
C:\Program Files\App\
C:\Windows\System32\
C:\Users\Public\
If the application searches directories that are writable by users, an attacker may place a malicious file named backup.exe in one of those directories. When the program runs, it may execute the attacker's malicious file instead of the legitimate one.
This vulnerability is also known as Path Injection or Search Path Hijacking.

Business Impact

Untrusted Search Path vulnerabilities can have serious consequences for organizations.
System Compromise
Attackers may execute malicious programs on critical systems.
Data Breach
Malicious software may access confidential data such as customer records or financial information.
Malware Installation
Attackers may install backdoors, spyware, or ransomware.
Operational Disruption
Compromised systems may experience downtime or malfunction.
Financial Loss
Organizations may incur costs related to incident response, recovery, and legal consequences.
Reputation Damage
Security incidents can reduce customer trust and harm the organization's reputation.

9. Vulnerability name: Information Loss or Omission

CWE: 221

OWASP category: A09:2025 Security Logging & Alerting Failures

Description

Information Loss or Omission is a security vulnerability that occurs when an application fails to record, store, or transmit important information properly. This may happen due to poor system design, incomplete logging, missing data handling, or errors in the software. As a result, critical information may be lost, omitted, or not properly tracked, which can affect system security, auditing, and decision-making.
In many systems, applications must handle large amounts of data such as:

- User activity logs
- Transaction records
- Error reports
- System monitoring data
- Security events

If the system does not properly capture or store this information, important events may go unnoticed.
For example, a system may record user login attempts. If failed login attempts are not logged correctly, administrators may not detect brute-force attacks or unauthorized access attempts.
Example of expected logging:
User: admin
Login attempt: Failed
Time: 10:45 AM
IP Address: 192.168.1.15
If the system fails to record this information, security teams lose visibility into suspicious activity.
Information loss may also occur when data is accidentally overwritten, deleted, truncated, or not transmitted correctly between system components.

10. Vulnerability name: Failure to Handle Missing Parameter

CWE: 287

OWASP category: A10:2025 Mishandling of Exceptional Conditions

Description

Failure to Handle Missing Parameter is a software vulnerability that occurs when an application does not properly check whether required input parameters are present before processing a request. Parameters are pieces of data that applications receive from users, forms, APIs, or URLs to perform specific operations.
In many web applications, parameters are passed through:

- URL query strings
- Form submissions
- API requests
- Cookies or headers

If an application assumes that a parameter will always be present and does not validate it, the system may behave incorrectly when the parameter is missing. This can lead to errors, unexpected behavior, system crashes, or security vulnerabilities.
For example, consider a web application that displays a user profile based on a user ID provided in the URL.
Normal request:
https://example.com/profile?user_id=123
The application uses the user_id parameter to retrieve the user's information. If the parameter is missing: https://example.com/profile and the application does not handle this case properly, it may produce an error or expose internal system information.

Business Impact

This vulnerability can negatively affect organizations in multiple ways.
Service Disruption
Application errors caused by missing parameters may interrupt services.
Poor User Experience
Users may encounter error pages or broken functionality.
Security Risks
Attackers may use these weaknesses to identify deeper vulnerabilities.
Operational Costs
Organizations may spend additional resources fixing application errors and handling user complaints.
Reputation Damage
Frequent system errors may reduce user trust in the application.

STAGE -2

OVERVIEW:-

An overview of a Nessus scan typically involves several important steps and aspects that help identify vulnerabilities in a computer system or network. Nessus is one of the most widely used vulnerability scanning tools developed by Tenable. It is mainly used by security professionals to detect security weaknesses, misconfigurations, and potential threats in systems before attackers can exploit them.

1. **Target Identification**

The first step in a Nessus scan is identifying the target system or network that needs to be scanned. The target can be a single IP address, a range of IP addresses, or a domain name.
For example, a tester may scan a web server, internal network, or a specific website to find vulnerabilities.

2. **Scan Configuration**

Before starting the scan, the user must configure the scan settings. This includes selecting the type of scan, such as a basic network scan, web application scan, or advanced vulnerability scan. The user can also configure credentials, ports to scan, and the level of scanning intensity. Proper configuration ensures accurate vulnerability detection.

3. **Plugin-Based Vulnerability Checks**

Nessus uses thousands of plugins to check systems for known vulnerabilities. These plugins contain information about security issues, outdated software, missing patches, and misconfigurations. During the scan, Nessus runs these plugins against the target system to detect possible weaknesses.

4. **Vulnerability Detection**

Once the scan begins, Nessus actively examines the target system. It checks open ports, running services, software versions, and system configurations. By comparing the collected information with known vulnerability databases, Nessus identifies security risks such as outdated software, weak encryption, and exposed services.

5. **Risk Assessment**

After detecting vulnerabilities, Nessus categorizes them based on severity levels, such as: • Critical

- High
- Medium
- Low
- Informational

This classification helps security teams prioritize which vulnerabilities should be fixed first.

6. **Report Generation**

After the scan is completed, Nessus generates a detailed report. The report contains:

- List of detected vulnerabilities
- Severity level of each vulnerability
- Description of the issue
- Possible impact on the system
- Recommended solutions or patches

These reports help administrators understand the security posture of their systems.

7. **Remediation and Re-Scanning**

The final step is fixing the identified vulnerabilities by applying security patches, updating software, or changing configurations. After remediation, another scan is usually performed to verify that the vulnerabilities have been resolved.

Nessus scan is an essential process in cybersecurity that helps organizations identify, analyze, and mitigate vulnerabilities in their systems. By following steps such as target identification, scan configuration, vulnerability detection, risk assessment, and reporting, Nessus helps maintain a secure IT environment and reduces the chances of cyberattack**s.**

1. A Nessus scan starts by identifying the target system, IP address, or website that needs to be tested for security vulnerabilities.
2. The user then configures the scan settings, including scan type, ports to scan, and authentication details if required.
3. Nessus uses a large database of vulnerability plugins to check systems for known security issues and weaknesses.
4. During the scanning process, the tool analyzes open ports, running services, and software versions on the target system.
5. The scanner compares the collected information with known vulnerability databases to detect potential security threats.
6. Identified vulnerabilities are then classified based on severity levels such as Critical, High, Medium, Low, and Informational.
7. Nessus provides a detailed description of each vulnerability, including the possible risk and impact on the system.
8. After completing the scan, the tool generates a comprehensive report listing all detected vulnerabilities and security issues.
9. The report also includes recommended solutions and remediation steps to fix the detected vulnerabilities.
10. Finally, security teams apply patches or configuration changes and perform a re-scan to ensure the vulnerabilities have been successfully resolved.

S.No	**Vulnerability Name**	**Security Severity**	**Plugin Name**
1	SSL Certificate Expired	High	SSL Certificate Expiry Plugin
2	Missing Security Patches	Critical	Microsoft Missing Security Updates Plugin

3	Open SSH Port (Port 22)	Medium	SSH Server Detection Plugin
4	HTTP Server Information Disclosure	Medium	Web Server Information Disclosure Plugin
5	Weak SSL/TLS Cipher Suites	High	SSL Weak Cipher Detection Plugin
6	Outdated Apache Web Server	High	Apache Version Detection Plugin
7	FTP Anonymous Login Enabled	High	FTP Anonymous Login Check Plugin
8	SMB Signing Not Required	Medium	SMB Signing Check Plugin
9	DNS Server Cache Poisoning	Medium	DNS Cache Poisoning Detection Plugin
10	ICMP Timestamp Request Enabled	Low	ICMP Timestamp Detection Plugin

TARGET WEBSITE: http://testfire.net/
TARGET IP ADDRESES: 65.61.137.117

REPORT:

1. SSL Certificate Expired SEVERITY: High

PLUGIN: SSL Certificate Expiry Plugin

PORT:443

DESCRIPTION:

This vulnerability occurs when the SSL certificate used by the web server has expired. An expired certificate means the encrypted connection between the user and the server is no longer trusted. Attackers may exploit this situation to intercept sensitive information transmitted between the client and the server.

SOLUTION:
Renew the SSL certificate from a trusted Certificate Authority and install the updated certificate on the server.

BUSINESS IMPACT:
If not fixed, users may receive browser security warnings, leading to loss of trust, reduced website traffic, and potential exposure of sensitive data.

2. Missing Security Patches

SEVERITY:Critical

PLUGIN: Missing Security Updates Plugin
PORT : 80

DESCRIPTION:

The system is missing important security updates and patches. Outdated systems may contain known vulnerabilities that attackers can exploit to gain unauthorized access or disrupt services.

SOLUTION:
Regularly update the operating system and installed software with the latest security patches.

BUSINESS IMPACT:
Failure to apply patches can lead to system compromise, data breaches, and operational downtime.

3. Open SSH Port
SEVERITY:Medium

PLUGIN: SSH Server Detection Plugin

PORT :22

DESCRIPTION:

The SSH service is detected running on the server. If not properly configured, attackers may attempt brute force attacks to gain access to the system.
SOLUTION:
Restrict SSH access using firewall rules, use strong passwords, and enable multi-factor authentication.

BUSINESS IMPACT:
Unauthorized access to the server may allow attackers to control the system and access confidential information.

4. HTTP Information Disclosure SEVERITY:Medium

PLUGIN: Web Server Information Disclosure Plugin

PORT : 80

DESCRIPTION:

The web server reveals information about the software and version it is running.
This information can help attackers identify known vulnerabilities in that software.
SOLUTION:
Configure the web server to hide version information and unnecessary details from HTTP headers.

BUSINESS IMPACT:
Attackers can use this information to plan targeted attacks against the system.

5. Weak SSL/TLS Cipher Suites
SEVERITY:High

PLUGIN: SSL Weak Cipher Detection Plugin

PORT :443

DESCRIPTION:

The server supports weak encryption algorithms. Weak cipher suites can allow attackers to decrypt or manipulate sensitive data transmitted over the network.
Solution:
Disable weak cipher suites and enable only strong and modern encryption protocols.

BUSINESS IMPACT:
Sensitive user information such as login credentials or personal data may be exposed during transmission.

6. Outdated Apache Web Server SEVERITY:High

PLUGIN: Apache Version Detection Plugin

PORT :80

DESCRIPTION:

The web server is running an outdated version of the Apache server. Older versions may contain known security vulnerabilities that can be exploited by attackers.

SOLUTION:
Update the Apache web server to the latest stable version and apply all available security patches.

BUSINESS IMPACT:
Attackers may exploit known vulnerabilities to gain unauthorized access or disrupt the website.

7. FTP Anonymous Login Enabled SEVERITY:High

PLUGIN: FTP Anonymous Login Check Plugin

PORT :21

DESCRIPTION:

The FTP server allows anonymous login without authentication. This allows anyone to access files stored on the server.

SOLUTION:
Disable anonymous login and require authentication for all FTP users.

BUSINESS IMPACT:
Unauthorized users may access, modify, or download sensitive files from the server.

8. SMB Signing Not Required SEVERITY:Medium

PLUGIN: SMB Signing Check Plugin

PORT : 445

DESCRIPTION:

The SMB service does not require message signing. Without signing, attackers may perform man-in-the-middle attacks and manipulate communication between systems.

SOLUTION:
Enable SMB signing on both client and server systems to ensure secure communication.

BUSINESS IMPACT:
Attackers may intercept or alter sensitive network communications.

9. DNS Cache Poisoning SEVERITY:Medium

PLUGIN: DNS Cache Poisoning Detection Plugin
PORT :53

DESCRIPTION:

DNS cache poisoning occurs when attackers insert false information into the DNS cache. This may redirect users to malicious websites without their knowledge.
SOLUTION:
Implement secure DNS configurations and regularly update DNS software.

BUSINESS IMPACT:
Users may be redirected to fraudulent websites, leading to data theft and loss of trust.

10. ICMP Timestamp Request Enabled SEVERITY:Low

PLUGIN: ICMP Timestamp Detection Plugin

PORT :ICMP

DESCRIPTION:
The system responds to ICMP timestamp requests. This allows attackers to determine the system time and potentially gather information useful for further attacks.
SOLUTION:
Disable ICMP timestamp responses in the firewall or system configuration.

BUSINESS IMPACT:
Although the risk is low, attackers may gather system information that could assist in more advanced attacks.

S.No	Vulnerability Name	Severity	Plugins
1	SSL Certificate Expiry Check	Medium	SSL Certificate Expiry Plugin
2	TLS Version 1.0 Protocol Detection	Medium	TLS Version Detection Plugin
3	HTTP Server Information Disclosure	Low	Web Server Information Disclosure Plugin
4	Missing Security Headers	Medium	HTTP Security Header Check
			Plugin
5	Open Port Detection	Low	Service Detection Plugin
6	DNS Server Detection	Informational	DNS Server Detection Plugin

TARGET WEBSITE : vit.ac.in TARGET IP ADDRESS : 122.184.65.22

REPORT:
1. SSL CERTIFICATE EXPIRY CHECK

SEVERITY: Medium

PLUGIN: SSL Certificate Expiry Plugin

PORT: 443

DESCRIPTION:
This vulnerability indicates that the SSL certificate used by the website may expire soon or is not properly configured. SSL certificates are responsible for encrypting communication between the user's browser and the web server. When the certificate becomes expired, the encrypted connection is no longer trusted by browsers. This may lead to security warnings when users try to access the website. Attackers may also take advantage of this situation to intercept communication between the user and the server.

SOLUTION:
The SSL certificate should be renewed before its expiration date through a trusted Certificate Authority. Administrators should implement automatic certificate renewal systems and regularly monitor certificate validity. It is also recommended to enable strong encryption protocols such as TLS 1.2 or TLS 1.3 and configure the web server to maintain updated security settings.

BUSINESS IMPACT:
If the SSL certificate expires, users may see browser warnings indicating that the website is not secure. This can significantly reduce user trust and may cause visitors to leave the website immediately. For educational or organizational websites such as Vellore Institute of Technology, this could affect online services such as admissions, student portals, and information systems. In severe cases, attackers may exploit the insecure connection to intercept sensitive data such as login credentials or personal information.

2. TLS VERSION 1.0 PROTOCOL DETECTION

SEVERITY: Medium

PLUGIN: TLS Version Detection Plugin

PORT: 443

DESCRIPTION:
The scan detected that the server supports older encryption protocols such as TLS 1.0. These older versions of TLS are considered insecure because they contain known vulnerabilities. Modern security standards recommend disabling outdated protocols since attackers may exploit weaknesses in these protocols to decrypt sensitive information transmitted over the network.
SOLUTION:
Administrators should disable TLS 1.0 and TLS 1.1 protocols on the server and enable only secure versions such as TLS 1.2 or TLS 1.3. The web server configuration should be updated to support strong encryption algorithms and secure cipher suites. Regular security audits and vulnerability scans should also be performed to ensure that outdated protocols are not enabled in the future.

BUSINESS IMPACT:
Allowing outdated encryption protocols increases the risk of cyberattacks such as man-in-the-middle attacks. Attackers may intercept and analyze encrypted communications to gain access to confidential information. For organizations providing online services, this can result in exposure of sensitive user data, reputational damage, and loss of user confidence. Maintaining strong encryption is essential for protecting online transactions and secure communication.

3. HTTP SERVER INFORMATION DISCLOSURE

SEVERITY: Low

PLUGIN: Web Server Information Disclosure Plugin

PORT: 80

DESCRIPTION:
This vulnerability occurs when the web server reveals information about the software and version it is running through HTTP response headers. When such details are exposed, attackers can easily identify the technologies used by the website and search for known vulnerabilities related to those versions.

SOLUTION:
The server configuration should be modified to hide unnecessary information from HTTP headers. Administrators should disable server version disclosure and remove details that reveal system configuration. Regular updates of server software and monitoring of server responses can also help minimize information leakage.

BUSINESS IMPACT:
Although the severity of this vulnerability is low, it can assist attackers in gathering information about the system. With this knowledge, attackers may attempt targeted attacks based on known weaknesses of the detected software version. This increases the risk of further exploitation and could lead to unauthorized access or service disruption if other vulnerabilities are present.

4. MISSING HTTP SECURITY HEADERS

SEVERITY: Medium

PLUGIN: HTTP Security Header Check Plugin

PORT: 80

DESCRIPTION:
The scan detected that the web server is missing several important HTTP security headers such as Content Security Policy (CSP), X-Frame-Options, and X-ContentType-Options. These headers play an important role in protecting web applications from attacks such as clickjacking, cross-site scripting, and content injection.

SOLUTION:
Administrators should configure the web server to include recommended HTTP security headers. For example, enabling Content Security Policy helps restrict the sources from which scripts and other resources can be loaded. X-Frame-Options should be enabled to prevent clickjacking attacks, and X-Content-Type-Options should be used to stop browsers from interpreting files incorrectly. Regular web security testing should also be performed to ensure these protections remain active.

BUSINESS IMPACT:
Without proper security headers, the website becomes more vulnerable to various client-side attacks. Attackers may inject malicious scripts into the website, which can steal user data or redirect users to malicious pages. Such incidents may affect user experience, damage the reputation of the organization, and reduce confidence in the website's security. Implementing security headers is a simple but effective way to improve web application protection.

5. OPEN PORT DETECTION

SEVERITY: Low

PLUGIN: Service Detection Plugin

PORT: Multiple Ports

DESCRIPTION:
The scan detected several open ports on the server. Open ports allow network communication between services and users. However, if unnecessary ports remain open, they can provide entry points for attackers to probe and exploit vulnerable services.

SOLUTION:
System administrators should review all open ports and determine whether they are required for system functionality. Unnecessary ports should be closed using firewall configurations. Access to critical ports should be restricted to trusted networks only, and intrusion detection systems should be used to monitor suspicious activity.

BUSINESS IMPACT:
Leaving unnecessary ports open increases the attack surface of the system. Attackers can scan these ports to identify running services and attempt to exploit vulnerabilities in them. If a vulnerable service is compromised, it may lead to unauthorized access, data leakage, or service disruption. Proper port management helps reduce the chances of successful cyberattacks.

6. DNS SERVER DETECTION

SEVERITY: Informational

PLUGIN: DNS Server Detection Plugin

PORT: 53

DESCRIPTION:
The vulnerability scan identified that the server is running a DNS service. DNS services are responsible for translating domain names into IP addresses so that users can access websites easily. This detection does not necessarily indicate a vulnerability but provides information about the services running on the system.

SOLUTION:
Administrators should ensure that the DNS server is properly configured and protected against common attacks such as DNS spoofing and cache poisoning. Security best practices include restricting DNS recursion, implementing DNSSEC, and monitoring DNS traffic for unusual activity.

BUSINESS IMPACT:
Although this is an informational finding, an improperly configured DNS server can create security risks. Attackers may manipulate DNS responses to redirect users to malicious websites or intercept sensitive information. Ensuring secure DNS configuration helps maintain reliable and secure access to online services and protects users from redirection attacks.

Stage 3:

Title: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Zero Trust Network Access (ZTNA)

- Zero Trust Network Access (ZTNA) is a modern security approach that provides secure access to applications based on strict identity verification and security policies. In traditional network security models, once a user is authenticated through VPN, they often gain broad access to the internal network. This increases the risk of unauthorized access and cyber attacks. ZTNA solves this problem by following the "never trust, always verify" principle.
- In the ZTNA model, every user and device must be verified before accessing any application or resource. Access is granted based on several factors such as user identity, device security status, location, and predefined access policies. Instead of allowing access to the entire network, ZTNA connects users only to the specific applications they are authorized to use. This approach reduces the attack surface and prevents lateral movement within the network.

Zscaler Private Access (ZPA)

- **Zscaler Private Access (ZPA)** is a cloud-based security service that provides secure access to internal applications without using traditional VPN. It follows the **Zero Trust Architecture** model, where users are verified before they are allowed to access any application.
- ZPA also helps organizations support secure remote work by allowing employees to access applications from anywhere while keeping the internal network hidden from the internet. By implementing ZPA, companies can reduce security risks, prevent unauthorized access, and protect sensitive data more effectively.

Principle of Least Privilege

- The Principle of Least Privilege (PoLP) is an important security concept in cybersecurity. It means that a user, system, or application should be given only the minimum level of access or permissions needed to perform their required tasks. Users should not have access to resources or systems that are not related to their job responsibilities.
- The Principle of Least Privilege is widely used in modern security frameworks such as Zero Trust Architecture and security solutions like Zscaler Private Access, where users are allowed to access only specific applications based on their identity and role. Implementing this principle improves overall security and ensures better control over organizational resources.

Identity-Aware Access

- Identity-Aware Access is a security approach where access to applications and resources is granted based on the identity of the user. Instead of trusting users automatically, the system verifies who the user is before allowing access to any application or data, the system checks several factors such as user credentials, authentication details, device security status, and access policies. Only if the user's identity is verified and matches the defined policies will access be granted. This ensures that unauthorized users cannot access sensitive systems or information.
- Identity-aware access is commonly used in modern security models like Zero Trust Architecture and platforms such as Zscaler Private Access, where users are allowed to connect only to the applications they are authorized to use. This approach improves security by ensuring that every access request is verified and controlled based on user identity.

Application-Specific Access Policies

- Application-Specific Access Policies are security rules that control which users can access specific applications within an organization. Instead of giving users access to the entire network, these policies allow them to access only the particular applications required for their job roles.
- Application-Specific Access Policies are security rules that control which users can access specific applications within an organization. Instead of giving users access to the entire network, these policies allow them to access only the particular applications required for their job roles.

ZPA Architecture and Components

The architecture of **Zscaler Private Access (ZPA)** is designed to provide secure access to internal applications without exposing the organization's network to the internet. It follows the **Zero Trust Architecture** model, where users are verified and granted access only to specific applications based on defined security policies. The ZPA architecture consists of several important components that work together to ensure secure and controlled application access.

1. **ZPA Client Connector:** The Client Connector is installed on the user's device such as a laptop, desktop, or mobile phone. It helps establish a secure connection between the user and the ZPA cloud platform. When a user tries to access an application, the client connector sends authentication and device information for verification.

2. **ZPA App Connector:** The App Connector is deployed inside the organization's data center or cloud environment. It acts as a bridge between internal applications and the ZPA cloud service. The App Connector establishes an outbound connection to the ZPA cloud, ensuring that internal applications remain hidden from the public internet.

3. **ZPA Service Edge:** The Service Edge is part of the ZPA cloud platform that manages secure connections between users and applications. It evaluates access policies, verifies user identity, and securely routes traffic between the user and the application.
4. **Identity Provider (IdP):** An Identity Provider such as Microsoft Azure Active Directory or Okta is used to authenticate users. It confirms the user's identity before allowing access to applications through ZPA.
5. **Policy Engine:** The Policy Engine is responsible for enforcing access policies. It checks user identity, device security, and access rules to determine whether the user is allowed to access a particular application.
6. **Application Segment:** Application segments define which applications are accessible to specific users or groups. This ensures that users can access only the applications that are authorized for their roles.

Implementation Methodology

The implementation methodology describes the steps followed to deploy secure application access using Zscaler Private Access (ZPA). The main objective is to enforce secure access policies and ensure that only authorized users can access specific applications by following the Zero Trust Architecture model.

Phase 1: Requirement Analysis: In this stage, the organization analyzes the security requirements and identifies the internal applications that need secure access. The user roles, departments, and access requirements are studied to design proper access control policies.

Phase 2: ZPA Configuration: The Zscaler Private Access platform is configured to provide secure access to applications. This includes setting up the ZPA cloud service and preparing the environment for implementing Zero Trust access control.

Phase 3: Identity Provider Integration: An identity provider such as Microsoft Azure Active Directory or Okta is integrated with ZPA. This step ensures that user authentication is verified before granting access to any application.

Phase 4: App Connector Deployment: The App Connector is installed within the organization's internal network or cloud environment. It creates a secure connection between internal applications and the ZPA cloud service without exposing the applications to the internet.

Phase 5: Access Policy Creation: Access policies are created to control which users or groups can access specific applications. These policies are based on user identity, role, and security requirements to ensure least-privilege access.

Phase 6: Testing and Validation: After configuration, the system is tested to verify whether users can access only the authorized applications. Security checks are performed to ensure that unauthorized users cannot access restricted resources.

Phase 7: Monitoring and Maintenance: Once the system is implemented, continuous monitoring is performed to track user activity and detect any suspicious behavior. Regular updates and policy adjustments are made to maintain system security.

Tools and Technologies

The implementation of secure application access using Zscaler Private Access (ZPA) requires several tools and technologies that help in authentication, policy management, and secure connectivity. These technologies work together to implement the Zero Trust Architecture model and enforce secure access to internal applications.

- **Zscaler Private Access (ZPA):** ZPA is a cloud security platform that provides secure access to internal applications without exposing the entire network.
- **Identity Provider (IdP):** Identity providers like Microsoft Azure Active Directory or Okta authenticate users and verify their identity before allowing access.
- **ZPA Client Connector:** The Client Connector is installed on the user's device to create a secure connection with the ZPA cloud.
- **ZPA App Connector:** The App Connector securely connects internal applications in the organization's network to the ZPA cloud service.
- **Web Applications / Internal Applications:** These are the business applications that users access securely through ZPA based on authorization.
- **Networking and Security Policies:** Security policies control which users can access specific applications based on identity and role.

Integration with Identity Providers

Integration with Identity Providers allows Zscaler Private Access (ZPA) to authenticate and verify users before granting access to internal applications. Identity providers such as Microsoft Azure Active Directory or Okta are connected with ZPA to manage user identities and authentication. When a user tries to access an application, the identity provider verifies the user's login credentials and confirms their identity. After successful authentication, ZPA applies access policies and allows the user to connect only to the authorized applications. This integration improves security by ensuring that only verified users can access organizational resources.

Device Posture Checking

Device Posture Checking is a security process used to verify whether a user's device meets the required security standards before allowing access to applications. It ensures that only secure and trusted devices can connect to internal resources. This process is commonly implemented in security platforms like Zscaler Private Access (ZPA) to strengthen access control and protect organizational data.

- Antivirus status and update verification
- Disk encryption status check (e.g., BitLocker, FileVault)
- Operating system update and patch level verification
- Detection of jailbroken or rooted mobile devices

- Device security risk score from endpoint protection tools

Micro-tunneling and Application Visibility

Micro-tunneling and application visibility are important security features used in Zscaler Private Access (ZPA) to ensure secure and controlled access to applications. Micro-tunneling creates a secure, temporary connection between the user and a specific application instead of providing access to the entire network. Application visibility allows administrators to monitor and track which applications users are accessing within the organization. It provides clear insights into user activity, application usage, and access patterns. By combining micro-tunneling with application visibility, organizations can ensure secure communication, improve monitoring, and maintain better control over application access.

Threat Intelligence Integration

Threat Intelligence Integration is a security process that helps identify and prevent potential cyber threats by using real-time threat data. In Zscaler Private Access (ZPA), threat intelligence is integrated to analyze network traffic, detect malicious activities, and block suspicious access attempts. It uses information from global threat databases and security analytics to identify harmful IP addresses, malware, and other cyber threats. By integrating threat intelligence, organizations can proactively protect their applications and networks, reduce security risks, and ensure safer access to internal resources.

Incident Response in a Zscaler Private Access (ZPA) Environment

Incident response in a ZPA environment refers to the process of detecting, analyzing, and responding to security incidents that may affect application access or user activity. Since ZPA follows the Zero Trust Architecture model, every access request is monitored and verified. If suspicious behavior or unauthorized access attempts are detected, the system can quickly respond to prevent security breaches. Effective incident response helps organizations identify threats early, minimize damage, and maintain secure access to internal applications.

Types of Incident Response

1. Detection
Identifying suspicious activities or potential security threats in the system.

2. Analysis
Investigating the incident to understand the cause and impact.

3. Containment
Limiting the spread of the threat by restricting access or isolating affected systems.

4. Eradication
Removing the threat such as malware or unauthorized access from the system.

Understanding ZPA: Key Takeaways

- **Zero Trust Model –** ZPA follows the Zero Trust Architecture where no user or device is trusted by default.
- **Application-Level Access –** Users are connected only to specific applications instead of the entire network.
- **Identity-Based Authentication –** Access is granted based on verified user identity through identity providers.
- **Secure Remote Access –** Employees can securely access internal applications from anywhere without using traditional VPN.
- **Reduced Attack Surface –** Internal applications remain hidden from the public internet, improving security.
- **Policy-Based Control –** Administrators can define access policies based on user role, device status, and security requirements.

Conclusion:

Stage 1: From web application testing, I understood the importance of verifying the functionality, performance, and security of web-based applications before they are deployed. Testing helps identify bugs, vulnerabilities, and performance issues that could affect the user experience or compromise sensitive data. It also ensures that the application works correctly across different browsers, devices, and operating systems. Overall, web application testing plays a crucial role in improving the reliability, security, and quality of modern web applications.

Stage 2: From the vulnerability report generated by Nessus, I understood how security scanning tools help identify weaknesses in systems, networks, and applications. The report highlights different types of vulnerabilities, their severity levels, and possible solutions to fix them. By analyzing these findings, security teams can prioritize critical issues and take necessary actions to strengthen system security. This process helps organizations prevent cyber attacks and maintain a secure IT environment.

Stage 3: From the Security Operations Center (SOC) and Security Information and Event Management (SIEM) dashboards such as IBM QRadar, I understood how organizations monitor and analyze security events in real time. These dashboards collect and analyze logs from different systems to detect suspicious activities and potential cyber threats. Security analysts use this information to investigate incidents, respond to threats, and improve overall security monitoring. This helps organizations quickly identify attacks and protect their systems and data from security breaches.

Future Scope:

Stage 1: Future Scope of Web Application Testing

The future scope of web application testing is expanding rapidly as web technologies continue to evolve and organizations increasingly rely on digital platforms for their services. As applications become more complex, testing processes must also improve to ensure security, reliability, and performance. Advanced testing tools and automation technologies will play a significant role in identifying vulnerabilities and improving application quality.

- **Automation in Testing:** Automated testing tools will reduce manual effort and help testers identify bugs and vulnerabilities more efficiently.
- **AI-Based Testing Tools:** Artificial intelligence and machine learning will help analyze large amounts of code and detect errors or security issues quickly.
- **Cloud-Based Testing:** With the growth of cloud computing, web applications will be tested across distributed cloud environments for better scalability and performance.
- **Cross-Platform Testing:** Future applications must function properly across multiple browsers, devices, and operating systems, increasing the importance of compatibility testing.
- **Security-Focused Testing:** As cyber threats increase, security testing will become a major focus to protect applications from attacks such as SQL injection, cross-site scripting, and data breaches.

Stage 2: Future Scope of the Testing Process

The testing process will continue to evolve as organizations adopt modern software development practices and advanced security technologies. Future testing environments will focus on automation, continuous monitoring, and faster vulnerability detection to ensure secure and reliable applications.

- **Continuous Testing:** Integration of testing into the software development lifecycle will allow applications to be tested continuously during development.
- **Integration with DevOps:** Testing processes will become more closely integrated with DevOps practices to ensure faster and more reliable software delivery.
- **AI-Driven Vulnerability Detection:** AI-based tools will help detect complex vulnerabilities and security weaknesses that may be difficult for manual testers to identify.
- **Testing for Emerging Technologies:** As technologies such as IoT, cloud computing, and mobile platforms grow, testing processes will expand to secure these environments.
- **Improved Performance Testing:** Future testing will focus on ensuring applications can handle high traffic loads and maintain optimal performance.

Stage 3: Future Scope of SOC / SIEM

The future scope of **Security Operations Center (SOC)** and **Security Information and Event Management (SIEM)** is rapidly growing as organizations require stronger cybersecurity monitoring and faster threat detection. Modern SOC environments will focus on advanced analytics, automation, and intelligent threat detection to protect organizational systems and data. Security platforms such as **IBM QRadar** will continue to evolve by integrating artificial intelligence and real-time security analysis to improve incident detection and response.

- **AI-Based Threat Detection:** Using artificial intelligence and machine learning to analyze security logs and identify suspicious activities automatically.
- **Automated Incident Response:** Implementing automated security workflows that can quickly respond to detected threats and reduce manual intervention.
- **Cloud Security Monitoring:** Expanding SOC and SIEM capabilities to monitor security events across cloud environments and hybrid infrastructures.
- **Threat Intelligence Integration:** Integrating global threat intelligence feeds to detect emerging cyber threats and improve proactive defense.
- **Advanced Security Analytics:** Using big data analytics to identify patterns, anomalies, and potential security risks in large volumes of log data.
- **Improved Security Visibility:** Providing better dashboards and monitoring tools to help security analysts detect and respond to incidents more efficiently.

Topics Explored:

1. Web Testing
2. Vulnerability Scanning
3. Nessus
4. SOC
5. SIEM
6. QRadar
7. Threat Detection
8. Incident Response
9. Security Monitoring
10. Cybersecurity

Tools Explored:

- Nessus
- IBM QRadar
- Zscaler Private Access
- Web Browser
- Security Dashboards